

Who Authorizes Software Behavior?

Governing the AI-Native SDLC

Bhash Ganti

Independent Researcher

Camas, WA, USA

ORCID 0009-0007-3810-6520 · bachipeachy@gmail.com

ABSTRACT

As software projects become increasingly agent-mediated, implicit authorization fails: the lifecycle is gated repeatedly—merge, release, deployment—and those disjoint approvals are assumed to compose, a composition living in reviewers’ heads that requires producers to be few, known, and sharing tacit context. Agents break all three, and no single point determines what behavior a system is authorized to realize. Authorization must instead be an explicit, independently checkable property established during governed construction and carried in a sealed state that execution realizes but cannot amend. Three tests are reported: a reference implementation assembling seven governed domains under one governance surface; an independent realization built from a frozen specification, claiming only the surfaces it exercised; and mutation testing in which removing two required guards left demonstrations unaffected, while a discriminating mutation of declared routing failed as expected. These address realizability, specification sufficiency, and evidence adequacy, exposing a concrete limitation of conventional evidence.

INDEX TERMS

Artificial intelligence, authorization, compilers, formal specifications, runtime environment, software architecture, software development management, software engineering.

I. SITUATION: HOW SOFTWARE IS AUTHORIZED TODAY

A. The authorization problem: a structural gap in the software lifecycle

Implicit authorization, lifecycle-wide. The software development lifecycle (SDLC) relies on an implicit authorization model that is failing. When autonomous agents participate at any stage, a foundational question emerges: *what authorizes the resulting behavior to execute?* It spans the lifecycle — transformations, deployments of changed behavior, decisions in continuous integration and delivery (CI/CD) pipelines, version transitions and rollbacks — and concerns authority itself.

Distinct and unaddressed. Authorization—the power to decide which behaviors may exist at any lifecycle stage—differs from four properties that established mechanisms address well: *correctness* (does it match intent), *provenance* (where it came from), *identity* (what or whom it identifies), and *trust* (whether the producer or artifact is believed). These characterize an artifact and its production; *authorization governs which behaviors may exist and transform through the lifecycle*. A system can satisfy all four properties yet still fail to answer who authorized any particular behavior at any stage.

Gated approvals, assumed to compose. Conventional development does not rest on a single approval. It gates the lifecycle repeatedly—code review at merge, security review before release, a change advisory board before deployment, operational sign-off before a configuration takes effect. Each gate authorizes a transition, and none carries its determination past the next: the reviewer who approved a merge does not decide what the pipeline deploys, and the board that approved a deployment does not decide what a rollback restores. The gates are disjoint, and the lifecycle is governed only on the assumption that they compose. Three conditions made that assumption workable: producers were *few* (bounded by human capability), *known* (reviewers assessed their track record), and shared *tacit context*—unwritten organizational constraints enforced by present reviewers. Where these held, a human at a later gate could reconstruct what an earlier one had intended. The composition lived in reviewers’ heads rather than in the artifact.

B. Why implicit authorization fails across the lifecycle

Agent-mediated participation anywhere in the lifecycle structurally breaks all three preconditions: bounded scale, known producers, and shared tacit context. Scale no longer bounds producer output: computational capacity scales production orders of magnitude beyond what any human can

meaningfully authorize individually. No producer accrues persistent reputation for adherence to a given organization’s unwritten constraints. Tacit constraints—design patterns, security postures, regulatory obligations, deployment topology decisions—live in institutional memory; agents cannot adhere to what they cannot see.

The failure cascades across stages. CI/CD pipelines make deployment and runtime behavior decisions implicitly: which configurations to apply, what behavior changes to permit, what rollback conditions exist—all determined by pipeline logic rather than explicit authorization. Version management permits automated transitions without explicitly determining authorized behavior at each version. Security policies and admission controls constrain containers but not the behavior they realize. Configuration systems apply settings without declaring what behavior those settings authorize. The result is *unlocatability* at every stage: authorization events become indistinguishable from proposals, checks, approvals, and automated decisions.

The preconditions do not hold at any single stage, let alone across the lifecycle. The architectural response is clear: **make authorization an explicit artifact throughout the lifecycle, separable from production and independent of who or what produces at each stage.**

C. Existing mechanisms stop short of lifecycle-wide authorization

Supply-chain frameworks (SLSA [1], in-toto [2], Sigstore [3], reproducible builds [4]) establish build provenance precisely but not what behavior is authorized to result; a fully attested build of unauthorized code remains unauthorized. CI/CD systems and admission controls (Kubernetes admission controllers [5], OPA [6], SPIFFE/SPIRE [7]) constrain what enters execution boundaries and establish workload identity, but they do not provide lifecycle-wide authorization, determine all behavior once admitted, or validate which lifecycle transitions are authorized. Model-driven engineering (OMG MDA [8], BPMN [9]) specifies behavior declaratively at design time but typically treats models as code-generation inputs; generated code diverges from models, and downstream stages disconnect from design determination.

Each addresses a genuine property at a single lifecycle stage or boundary, and none makes authorization a first-class artifact the system carries and propagates. They leave the seams between stages ungoverned—precisely where development authorization fails to reach deployment, rollback,

or configuration change. The unanswered question is whether an executable system at any lifecycle point can explain what authorized its current behavior, and whether that explanation can be verified without relying on its producers. Table I sets these mechanisms side by side. This paper formulates authorization as a lifecycle property that persists with governed state, and reports a realization of it.

TABLE I. Where existing mechanisms stop.

Mechanism family	Lifecycle stage addressed	What it establishes	Authorization carried in the artifact	Behavior after admission
Supply-chain provenance [1]–[4]	construction	how an artifact was built	no	ungoverned
Admission and policy [5], [6]	deployment boundary	what may enter	no — policy is external to the artifact	ungoverned
Workload identity [7]	deployment, runtime	what or whom is running	no	ungoverned
Model-driven engineering [8], [9]	design	declarative behavioral models	no — models are code-generation inputs	ungoverned
Protocol-Governed Computing (this paper)	inception through evolution	what behavior is authorized	yes — in the sealed snapshot	governed — traversal of the sealed determination

II. TARGET: WHAT A RESOLVED SOFTWARE LIFECYCLE SHOULD LOOK LIKE

Authorization becomes an explicit, persistent, independently checkable property of the software lifecycle rather than an implicit property of the people and processes surrounding it. It must hold at every stage—inception, design, construction, admission, execution, evolution—so that it remains determinative of what may execute and what may replace it.

A reviewer should be able to take an executable state at any point in that lifecycle and answer the following from the system's own state, without relying on anyone who participated in producing it:

1. Who or what determined this behavior?
2. What declaration authorized that determination?
3. Where is that authorization carried?
4. Can the behavior execute without it?
5. When the behavior changes, what authorizes the new state?

Here, *who or what* identifies the determining authority rather than an individual actor. These questions are not the target; they measure whether it has been reached. The four properties of Section I-A describe an artifact; authority determines whether the behavior that artifact represents may exist and execute.

A lifecycle reaches the target when those five questions have inspectable answers at every stage where behavior is introduced or changed, held in the system's governed state rather than in the memory, judgment, or presence of whoever produced it.

This changes what execution means. **Execution no longer decides what the software is allowed to become; it realizes behavior that has already been determined and authorized.** It also changes what change means: modifying running software is not an informal continuation of development, since the next executable state must acquire authorization before it can execute. These consequences ensure the property holds across the entire lifecycle rather than at a single gate.

The target is neutral on language, compiler, runtime, deployment topology, and trust mechanism. Those are implementation choices; Section III makes one set, and a reader rejecting them should still use this section to judge any other candidate.

III. PROPOSAL: A GOVERNED SDLC

Protocol-Governed Computing (PGC) moves the primary governance determination out of execution and into governed construction, where it is incorporated into the sealed state that execution realizes.

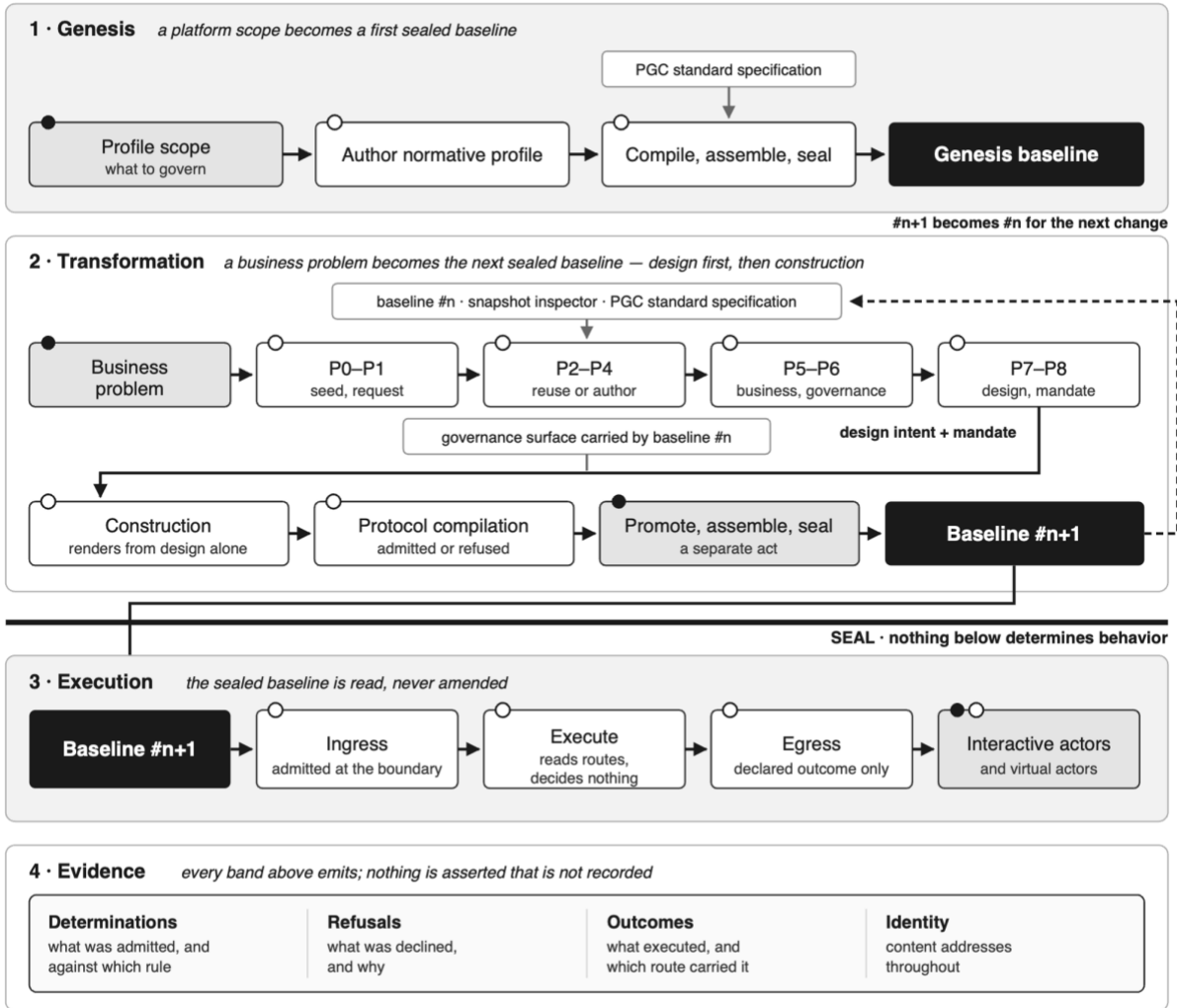
Determined before execution, resealed on change. Instead of asking a runtime to decide whether behavior is permissible, PGC constructs a complete, sealed representation of the behavior the system is permitted to realize. The runtime consumes that representation; it does not add behavior to it. When the system must change, the change is another governed construction that produces the next sealed representation. The whole lifecycle is one chain: *scope, determine, construct, seal, execute, observe, transform, reseal*.

Neutral to producer and domain. PGC does not require a particular kind of producer. Here *producer* denotes any mechanism performing a permitted lifecycle step — a human, an AI agent, or anything else; Section IV-B distinguishes the *profile author* and the *builder* as separate roles in the independent trial. The governing determination comes from the protocol and its declared inputs, not from the identity of the producer performing the step. Consequently, replacing the producer does not, by itself, change the determination. Execution is likewise domain-neutral: the runtime carries no domain knowledge, and behavior was fixed by compilation before execution.

Fig. 1 shows the complete lifecycle. The exposition below walks it in the figure’s own order, because that order is the argument: authority is established before anything runs, and what runs cannot enlarge it.

Protocol-Governed Computing — the governed lifecycle

● human act ○ agent may derive → consulted, not consumed



An agent may derive every open step. A human sets the scope, states the problem, and promotes — so proposing software and authorizing software remain separate acts.

Fig. 1. *The governed lifecycle.* Genesis takes a platform scope to a first sealed baseline; transformation takes a business problem to the next, design before construction; execution reads the sealed baseline and never amends it; evidence records what every band above determined. Filled circles mark the two human acts, open circles the steps an agent may derive. The heavy rule marks the **seal**: execution below it cannot determine new governed behavior.

A. Genesis

Genesis creates the first governed version of a system. A human first defines the scope—what system is being governed and which parts of the standard apply to it. A producer then authors that decision as a **normative profile**, in the form the standard specifies, and the profile is compiled and assembled into a first sealed **baseline**: the first complete representation the runtime is permitted to execute.

A normative profile is the system’s explicit selection among the options the standard [10] permits, together with the options it excludes. Its essential property is simple: **the profile cannot be authored by the system it governs**. Otherwise a system could choose its own governing rules and claim conformance to them. That is why the figure’s first act is human and is scope rather than design—what to govern is the decision that cannot be delegated to the system being governed. Concretely, a profile fixes a class of systems and decides everything the standard leaves open. It may be scoped to a single governed system: one accepted snapshot executed against governed state, with inspection enabled, no external protocol boundary, no additional attestation, one tenant, and no replication. Excluding the external boundary makes ingress behavior not merely unmade but unmakeable.

The resulting baseline is a **snapshot**: the governed system’s sealed state. *Sealed* means immutable from the moment it is constituted, identified by its own contents, and readable but not amendable by execution. It contains everything the runtime is permitted to need to execute and carries an identity derived from its own contents. *Baseline* names its position in the lifecycle—the state a system currently is and the one a transformation transforms; *snapshot* names the artifact and its properties. The important consequence is completeness: the runtime has no permitted role in inventing missing governed behavior. What the baseline contains is what execution can realize, and what it does not contain cannot be supplied at execution time as an ungoverned decision. The standard specification is *consulted, not consumed*: it constrains what a composition may claim but never enters the sealed state, so conformance is a property of what a system sealed rather than what it read.

B. Transformation

Genesis establishes the starting point. Transformation explains how the system changes while maintaining governance, and it is where the paper’s central separation becomes architectural rather than rhetorical.

A transformation begins with a business problem—the second human act—and grounds itself in baseline $\#n$, read through the snapshot inspector. The P0–P8 phases provide the governed path from that problem to a new design: P0–P1 take a plain-language problem statement to a change seed and change request; P2–P4 establish the domain model, analysis loop, and business model; P5–P6 separate business intent from governance intent, the point at which what the business wants and what the organization permits cease to be expressible as one statement; P7–P8 produce design intent and the **authoring mandate**, a complete and reviewable dossier. Each phase evaluates for admissibility against a declared rule set, and from P2 onward, consumes its priors, so handoffs between phases are checked rather than assumed.

The architectural ordering that matters is **design before construction**: the construction mechanism does not decide what the software should mean; it renders a design that has already been determined. The pipeline therefore separates two compilation problems. The first determines whether the design is admissible; the second determines whether that admissible design can be rendered into an executable artifact. They differ because their failures differ in kind: a design failure is an incomplete or contradictory mandate, caught by a phase’s rule set; a construction failure is a valid mandate that does not provide sufficient information to determine the required artifact. Only rendering exposes the latter, and repair amends the design language rather than a single register.

The rendered artifacts then pass through protocol compilation, where material violating the governing surface is **refused rather than silently repaired**. Promotion and sealing follow as separate acts, deliberately not fused to construction: producing a candidate system is not the same as authorizing it to become the next baseline. Construction therefore produces a candidate; promotion authorizes it; sealing makes that authorization part of the executable state. This is where the figure’s legend carries the thesis: *an agent may derive every open step; a human sets the scope, states the problem, and promotes—so proposing software and authorizing software remain separate acts*.

Once sealed, baseline $\#n+1$ becomes baseline $\#n$ for the next transformation. Evolution is another governed construction cycle, run against the state the previous cycle established rather than against a fresh reading of intent.

C. Execution

Execution is deliberately the simplest band in the figure. It reads the sealed baseline, admits at the boundary only what the baseline permits, follows the behavior already determined there, and emits only declared outcomes. It does not modify the baseline or make new governance decisions. No new governed behavior is determined at runtime; execution traverses what the snapshot already contains [11].

Two boundary cases test that claim. Encountering a state the baseline does not authorize — a corrupted snapshot, an unresolvable binding — execution refuses, and refusal is a declared outcome rather than a new determination. Rollback is a governed transformation to a previously sealed baseline, so it is available only where that baseline is retained; retention is an operational precondition the architecture requires but does not itself provide.

This resolves the potential misreading: **execution is governed, but governance is not determined there**. The runtime consumes the determination; it does not become the authority that made it. A behavior absent from the baseline is not merely unimplemented—there is no path by which execution could introduce it.

D. Evidence

Evidence makes governance observable. Each band above records what it determined: admissions and the rule each was admitted against, refusals and their causes, execution outcomes and the routes that carried them, and the content-derived identities of the governed artifacts throughout.

The purpose is not audit logging. The record enables a party who participated in neither construction nor execution to examine what was determined, what occurred, and which declared rules governed those outcomes—the property Section I identified as absent when authority distributes across pipelines and approvals. Refusal deserves particular emphasis, because most architectures never record it: a system logging only what it did cannot distinguish a guard that held from one that was never reached. Section IV turns on exactly that distinction.

E. What the architecture does not prescribe

The architecture governs the determination, not every detail of its realization. The profile selects what is governed and what is out of scope. Binding a declared capability to its handler is the implementer’s choice, subject to being declared and resolving before dispatch. Determinism binds the *determination*—the same sealed state admits the same behavior—not the language, runtime, or topology realizing it. It also assumes a trust boundary it does not establish: sealing provides integrity through content-derived identity, not signature; multi-party trust is recorded in Section VI as a limitation, not claimed as a property.

This architecture is not merely conceptual. The reference realization implements the lifecycle of Fig. 1 as specified by the Open PGC Standard family [10] and assembles seven governed domains under one governance surface, among them business domains as unrelated as AI governance, blockchain, and book library management [12]. That standard provides the detailed construction specification; this paper abstracts that machinery to expose the architectural argument and reports the realizations needed to test it. Conceptual foundations appear in [13], compilation in [14], execution in [11], the reorientation in [15], and the constitutional model constraining autonomous and AI-generated software in [16] — published under the architecture’s former name, Protocol-Governed Systems.

IV. EVIDENCE: WHAT HAS BEEN BUILT AND WHAT IT ESTABLISHES

Three distinct questions can be asked of this architecture. Can a nontrivial system actually be built this way? Can someone else build it from the specification and its declared profile, without access to an existing realization? And do the demonstrations establishing those results establish what they claim to? The sections below report them separately, so the strongest evidence does not borrow credibility from the broadest. The prior corpus cited in Section III-E documents the architecture and its development. This paper reports evidence rather than restating that corpus: it assumes the reader can consult the standard [10], and the realization and validation activities below are the contribution.

A. Realizability

The reference implementation [12] assembles seven governed domains comprising 410 protocol artifacts under a single governance surface. That establishes two bounded results: a nontrivial system can be built on the model, and the same governance surface can accommodate the seven domains exercised without domain-specific governance machinery. This is evidence of realizability, not independent confirmation of the model, because the standard and the reference implementation share an author.

B. Independent realization

The second test asked whether a realization could be derived from the specification without access to the reference implementation. A separate authoring run produced a normative profile from the frozen specification [10] alone. A separate builder, given that profile and the specification but no reference-implementation material, then constructed a realization claiming the profile, drove its execution from sealed declarations, and performed a governed transformation against its own baseline. The independence claimed here is implementation independence, not independence of all prior authoring context; Section VI states the remaining contamination boundary.

Three of the profile’s eight claims were discharged: the vocabulary and declaration surface, construction and transformation, and runtime and execution. The remaining five are not failures, and the distinction matters. *Discharged* means an evidence document names the claim and supplies

demonstrations that fail when the behavior is removed. Three further claims — snapshot conformance, evidence, and inspection — were *exercised*, the machinery built and used, but no claim was made for them; declining to claim is correct behavior. The last two, profile conformance and system instance, were not claimed at all because the realization did not exercise the surface they require. Those surfaces remain unvalidated rather than failed.

The count is a property of the evidence, not a score. A realization claiming all eight would be the failure this paper warns of, since a claim discharged by a demonstration that cannot fail establishes nothing (Section IV-C). Declining the five is that discipline applied to the builder’s own work.

One further result bears on sufficiency: across three separately conducted profile-authoring runs, seventeen candidate findings were raised and none was an undeclared gap — every one landed on a decision the standard had already marked as deliberately left open. This is the strongest evidence produced by this cycle about specification sufficiency, bounded to the surfaces actually exercised. The criterion is stated so the result can fail: a finding falsifies sufficiency when it names a decision the standard neither determines nor marks as deliberately open, so that the author had to supply the answer from outside the specification. No finding in this cycle met that criterion.

C. Evidence adequacy

A system can pass every demonstration written for it while those demonstrations fail to establish the property they purport to test. Five mutations were applied. The first two removed guards that were required by the profile and correctly implemented, to see what would detect the change. A required SHA-256 digest was replaced with MD5, and all six demonstrations written for it still passed. A baseline-grounding guard was disabled entirely, and all fifteen still passed.

Three possibilities are worth naming. A **non-discriminating** demonstration lets both the conforming implementation and a violating mutation pass. A **discriminating** demonstration makes the mutation fail. A property merely *asserted*, with no demonstration capable of separating the two, has not been evidenced at all. Both guards above fall in the first category: demonstrations existed and passed, but none could have failed.

Here, mutation testing tests evidence adequacy [17], [18]: whether a claimed normative guard is detectable when removed, rather than whether a suite detects a representative fault population. The

experiment was risk-directed rather than systematic; no mutation score is therefore reported. The results establish the discriminating adequacy of specific demonstrations, not mutation coverage of the suite.

A mutation is correspondingly discriminating when the existing and mutated implementations agree on the tested conforming inputs but diverge under a demonstration specifically capable of exercising the removed or altered property. Declared routing is exactly such a case: reading a route from the sealed declaration and hard-coding the equivalent branch produce identical results on every well-formed input. That mutation fails, as do emptying the declared routes map and removing either refusal path (Table II). Its failure is evidence that this realization reads the tested behavior from what was sealed, rather than reproducing that behavior independently in the runtime.

The generalizing finding is not about carelessness. When the risk was stated explicitly — *could routing be hard-coded without detection?* — the discriminating demonstration was constructed. When that risk remained implicit, existing evidence did not expose it in either case. **A passing suite is not evidence that a demonstration could fail.** No existing suite test, inspection, or evidence document made the earlier gaps visible. This is why Section III-D’s refusal record is load-bearing, and it is the basis of the obligation Section V makes explicit.

TABLE II. Mutations.

Mutation	Demonstrations	Outcome	What it establishes
Required SHA-256 digest replaced with MD5	6	all passed	non-discriminating — a required property never demonstrated
Baseline-grounding guard disabled	15	all passed	non-discriminating — a correct guard asserted about, never exercised
Declared routing replaced with equivalent hard-coded branch	22-test suite	failed	discriminating — behavior demonstrably read from sealed state
Declared routes map emptied	22-test suite	failed	declared routing is load-bearing, not incidental
Either declared refusal path removed	22-test suite	failed	refusal is demonstrated rather than asserted

V. IMPLICATIONS: WHAT FOLLOWS FROM THE TARGET

Section II's target — authority as an explicit property of a system's own state, checkable by someone who was not present when it was established — carries three consequences, and a fourth that follows from how those consequences must be evidenced.

The producer does not confer authority. Once authorization lives in state rather than in an approving act, who or what proposed a change is no longer determinative of permissibility. An agent may derive every open step of a lifecycle without acquiring the power to decide what the system may do [16].

Evidence carried in state is independently checkable. It can be read directly by a party who did not participate in producing it, without reconstructing process claims from records never designed to answer the question [10].

Implementation replacement need not change governed behavior. Once determination separates from implementation, the implementation is no longer the authoritative statement of what the system does; the determination outlives the artifact realizing it [15].

Normative claims must be falsifiable. For each claimed normative guard, evidence must identify a demonstration whose outcome changes when the guard is removed or bypassed. As Section IV-C shows, a passing suite is not evidence that a demonstration could fail.

VI. LIMITATIONS AND FUTURE WORK

Section IV’s evidence is bounded by deliberate scope choices, stated here so that the claims in Sections II through V rest on explicit foundations. That evidence is sufficient for the core claims about authorization as an explicit, checkable state property; the following boundaries limit generalization beyond them.

- **Independent realization was exercised on the vocabulary, transformation, and execution surfaces.** A single author performed specification-based realization for this cycle. For the surfaces exercised — declaration, compilation, and runtime behavior — this single realization demonstrates that the specification itself contains sufficient detail to ground a separate implementation. A different author would strengthen this result; this cycle provides no basis for predicting whether such an author would obtain identical findings or whether the gaps surfaced would differ. This is why Section IV-B claims surfaces *discharged*, *exercised*, or *not claimed* rather than claiming profile conformance overall.
- **Comparative conformance was deliberately out of scope.** The authored profile was designed to avoid the reference implementation, blocking direct comparison by design rather than by implementation failure. Testing comparative conformance would require a profile narrow enough for both implementations—a separate design exercise suited to future work.
- **External effects and system boundaries were deliberately excluded.** The profile selects no interaction boundary, so no capability producing effects outside the sealed system was exercised. That path remains untested rather than failed.
- **One binding resolution remains implementation-specific.** The standard [10] requires declared bindings; the realization uses implicit binding on declared effect values. This is a known deviation. Declared binding artifacts are future work.
- **Single-node, unsigned, locally sealed.** Sealing uses content-derived identity, not cryptographic signature. Distributed, multi-party governance is not exercised and should not be inferred from this evidence.

- **Overhead and cost are not measured.** No performance or resource data are reported for compilation, sealing, or governed execution. Cost analysis is future work.
- **Conditional, cascading, and revocable authorization were not exercised.** Authorization here is established in sealed state. Conditional authorization on runtime context, cascading authorization, and revocation while running lie outside the exercised surface and are not claimed.
- **Two questions about the model itself are unresolved.** Whether protocol-governed computing is a distinct architectural property or a disciplined composition of the Section I-C mechanisms sealed into one model; and whether declarative determination stays expressive enough to govern real behavior without becoming another programming language.
- **The architecture establishes governance, not quality.** Authorization determines what behavior is permitted; it does not determine the quality of that behavior.

These boundaries define the surfaces on which Sections II through V make claims. Extending to multi-party governance, measuring overhead, exercising external effects, and declaring bindings would require separate cycles, each producing its own evidence. Adoption conditions are not evaluated here: the evidence establishes architectural properties, not migration cost, operational suitability, or comparative economics. What the architecture changes is where judgment is exercised: making governance decisions explicit and sealing them into state, rather than leaving them implicit in process and personnel.

VII. CONCLUSION

This paper argues that authorization should be an explicit, persistent property of governed state, established separately from the mechanism that realizes it. When production is agent-mediated, that separation keeps proposing software and authorizing software as distinct acts: a producer may derive behavior but cannot thereby authorize it.

Three forms of support carry that argument, and they answer to different standards of evidence. The architecture states the case for authorization as an explicit lifecycle property and rests on reasoning rather than implementation. The reference implementation establishes feasibility: seven governed domains under one governance surface. The Open PGC Standard [10] captures the model as an implementation-independent specification, and Section IV reports what a separate realization built from it discharged and declined. **Evidence bounding one form of support does not bound the others.**

The evidence also exposes a limitation of conventional evidence: demonstrations can pass while establishing nothing unless built to fail when a claimed guard is removed. That finding holds however far the rest of the validation reaches.

The result is narrow and stated so it can be tested: authorization can be made an explicit, independently checkable property of governed state, with execution realizing prior determination rather than becoming its source. Section VI names the surfaces not exercised; the consequences in Section V follow from that separation as architectural claims awaiting evaluation across topologies beyond those tested.

ACKNOWLEDGMENT

The author used Claude Opus 5 (Anthropic) for copyediting and critical review throughout, to improve the clarity and precision of the author’s exposition, and to render Fig. 1 from the author’s design. The author reviewed and approved the final text and figure, and takes full responsibility. The core technical concepts, paper structure (the STP framework), and all technical assertions and conclusions are the author’s own.

REFERENCES

- [1] Open Source Security Foundation, “Supply-chain levels for software artifacts (SLSA), specification v1.1.” [Online]. Available: <https://slsa.dev/>. Accessed: Aug. 31, 2026.
- [2] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola and J. Cappos, “in-toto: Providing farm-to-table guarantees for bits and bytes,” in *Proc. 28th USENIX Security Symp.*, Santa Clara, CA, USA, Aug. 2019, pp. 1393–1410. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias>
- [3] Z. Newman, J. S. Meyers and S. Torres-Arias, “Sigstore: Software signing for everybody,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, Los Angeles, CA, USA, Nov. 2022, pp. 2353–2367, <http://doi.org/10.1145/3548606.3560596>.
- [4] Reproducible Builds project. [Online]. Available: <https://reproducible-builds.org/>. Accessed: Aug. 31, 2026.
- [5] Kubernetes Authors, “Admission controllers,” Kubernetes Documentation. [Online]. Available: <https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/>. Accessed: Aug. 31, 2026.
- [6] Open Policy Agent, Cloud Native Computing Foundation. [Online]. Available: <https://www.openpolicyagent.org/>. Accessed: Aug. 31, 2026.
- [7] SPIFFE and SPIRE, Cloud Native Computing Foundation. [Online]. Available: <https://spiffe.io/>. Accessed: Aug. 31, 2026.
- [8] Object Management Group, *MDA Guide*, rev. 2.0, OMG document ormsc/14-06-01, Jun. 2014. [Online]. Available: <https://www.omg.org/mda/>. Accessed: Aug. 31, 2026.
- [9] Object Management Group, *Business Process Model and Notation (BPMN)*, version 2.0.2, OMG standard, Dec. 2013. [Online]. Available: <https://www.omg.org/spec/BPMN/2.0.2/>. Accessed: Aug. 31, 2026.

- [10] B. Ganti, *Open Protocol-Governed Computing Standard*, version v0, Zenodo, 2026, doi: <https://doi.org/10.5281/zenodo.22150616>.
- [11] B. Ganti, “Protocol-governed systems: Runtime conceptual model,” Zenodo, 2026, doi: <https://doi.org/10.5281/zenodo.20478471>.
- [12] B. Ganti, *Protocol-Governed Computing: Reference Implementation (Composed Platform)*, version v2, Zenodo, 2026, <https://doi.org/10.5281/zenodo.22184748>.
- [13] B. Ganti, “Protocol-governed systems: A conceptual model,” Zenodo, 2026, doi: <https://doi.org/10.5281/zenodo.20300611>.
- [14] B. Ganti, “Protocol-governed systems: Compiler conceptual model,” Zenodo, 2026, doi: <https://doi.org/10.5281/zenodo.21882441>.
- [15] B. Ganti, “Protocol-governed systems: Architecture inversion concepts,” Zenodo, 2026, doi: <https://doi.org/10.5281/zenodo.21882642>.
- [16] B. Ganti, “Protocol-governed systems: A constitutionally constrained architecture for autonomous and AI-generated software,” Zenodo, 2026, <https://doi.org/10.5281/zenodo.20330650>.
- [17] Y. Jia and M. Harman, “An analysis and survey of the development of mutation testing,” *IEEE Trans. Softw. Eng.*, vol. 37, no. 5, pp. 649–678, Sep. 2011, <https://doi.org/10.1109/TSE.2010.62>.
- [18] J. H. Andrews, L. C. Briand and Y. Labiche, “Is mutation an appropriate tool for testing experiments?” in *Proc. 27th Int. Conf. Software Engineering (ICSE)*, St. Louis, MO, USA, May 2005, pp. 402–411, <https://doi.org/10.1109/ICSE.2005.1553583>.

AUTHOR BIOGRAPHY

Bhash Ganti is an independent researcher in governed computing architecture. Current work concerns protocol-governed computing — making the authorization of software behavior an explicit, compiled, independently checkable property of the development lifecycle — and the conformance evidence required to establish it.

ORCID: <https://orcid.org/0009-0007-3810-6520>

GitHub: <https://github.com/bachipeachy>

Contact at: <mailto:bachipeachy@gmail.com>.